Software Analysis and Design

Lecturer : Ibrahim Manhawary

Team members : Nagham saqer , Maya Hasan , Nora Alkhayyat

Project Objectives:

- ★ Reduce order processing errors between waiters, kitchen staff, and delivery service
- ★ Improve communication through automated notifications and alerts.
- ★ Enable easy customization of orders (extras, special instructions).
- ★ Support multiple order types (dine-in, takeaway, delivery) within one unified system.

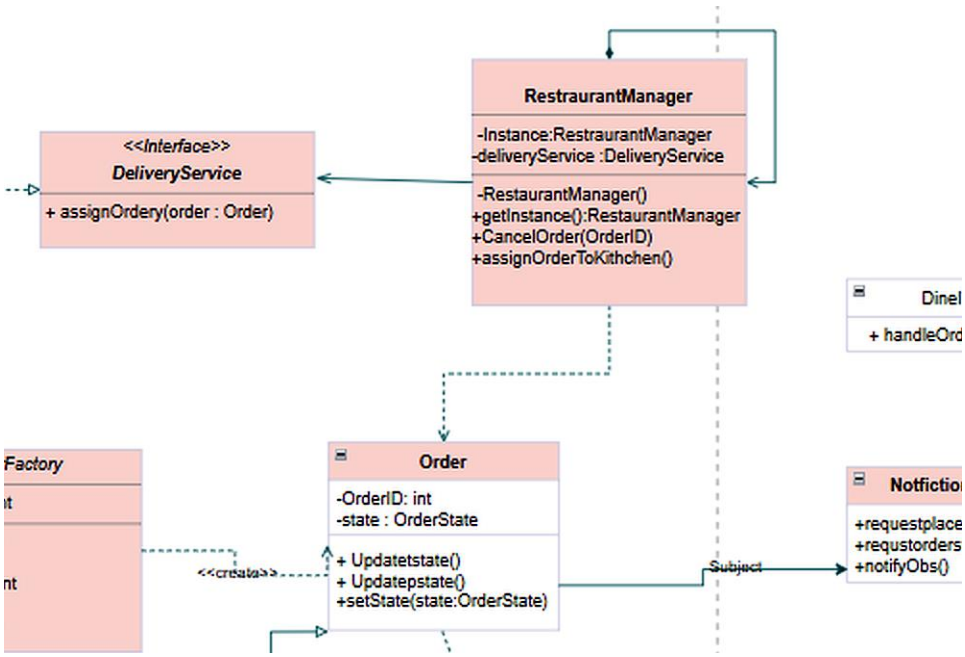
Application Scope:

The Restaurant Management & Ordering System (RMOS) is designed as a web-based application with mobile support. This type is suitable because it allows real-time access for restaurant staff and customers from different devices, ensuring flexibility, scalability, and ease of maintenance.

Application of Design Patterns

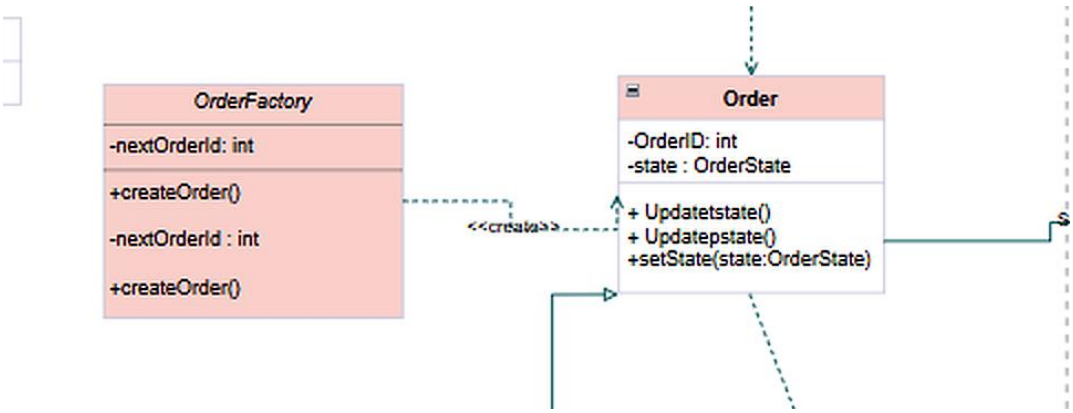
1. Singleton Pattern

- ★ **Role:** Ensures a single instance of `RestaurantManager` controls orders, tables, and menu data.
- ★ **Problem Solved:** Prevents inconsistent system states caused by multiple managers.
- ★ **SOLID Compliance:**
    - **OCP:** Extendable without modifying existing logic.



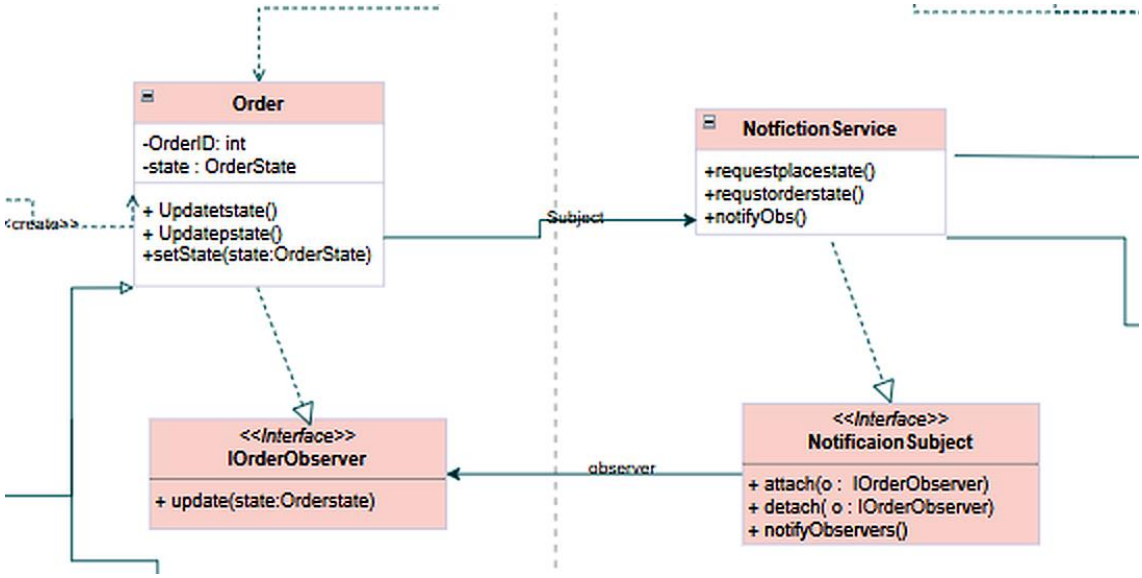
2. Factory Pattern

- ★ **Role:** `OrderFactory` creates different order types (Dine-in, Takeaway, Delivery).
- ★ **Problem Solved:** Encapsulates object creation and removes tight coupling.
- ★ **SOLID Compliance:**
  - **OCP:** New order types added without code modification.
  - **DIP:** Depends on the `Order` abstraction.



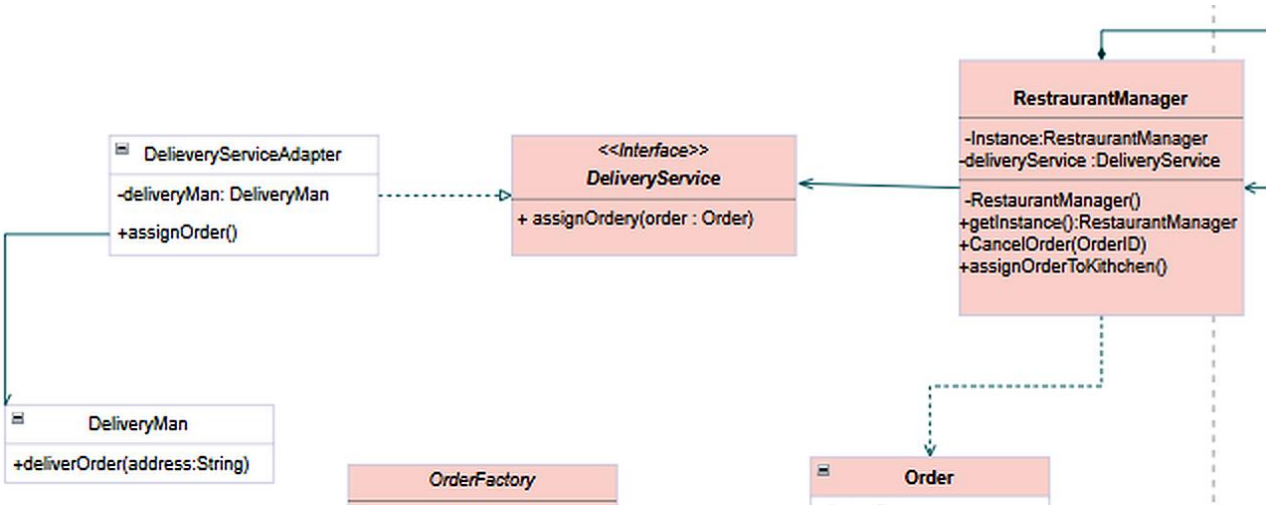
3. Observer Pattern

- ★ **Role:** `NotificationService` listens to changes in orders and tables.
- ★ **Problem Solved:** Enables real-time notifications with low coupling.
- ★ **SOLID Compliance:**
  - **OCP:** New notification channels added easily.



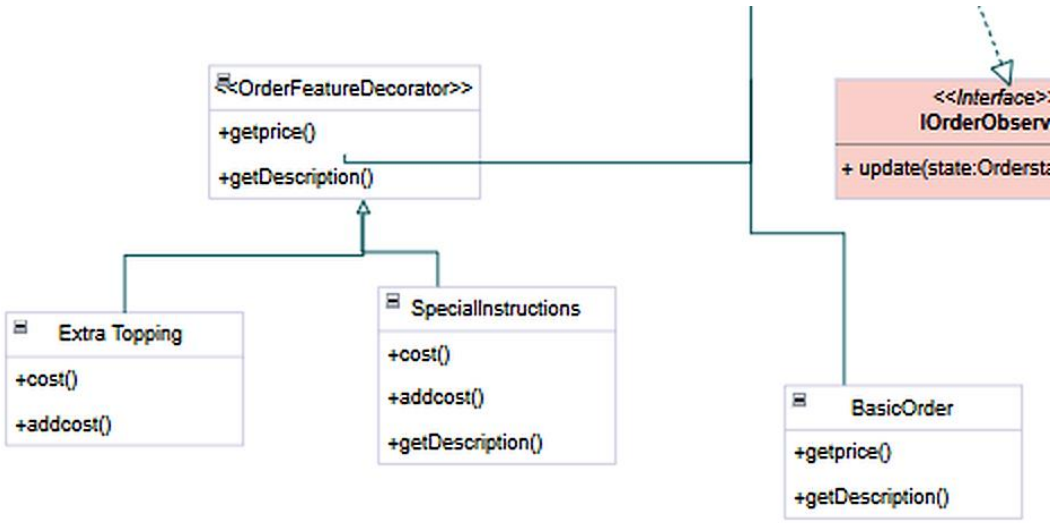
4. Adapter Pattern

- ★ **Role:** `DeliveryServiceAdapter` integrates external delivery platforms.
- ★ **Problem Solved:** Allows incompatible APIs to work with the system.
- ★ **SOLID Compliance:**
  - **OCP:** New adapters added without changing core logic.



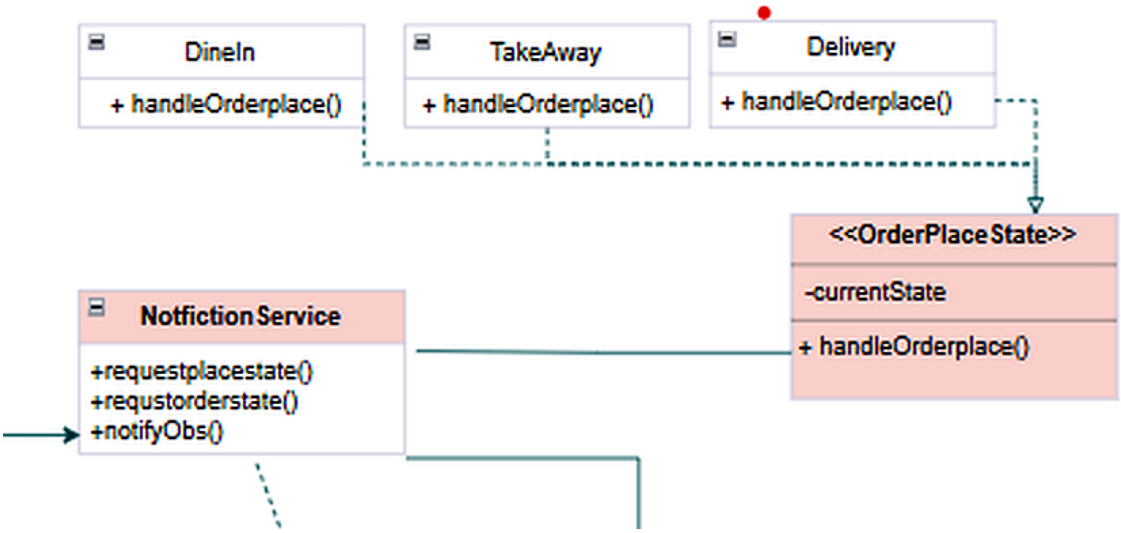
5. Decorator Pattern

- ★ **Role:** `OrderFeatureDecorator` adds optional order features dynamically.
- ★ **Problem Solved:** Avoids subclass explosion for order customizations.
- ★ **SOLID Compliance:**
  - **SRP:** Each decorator adds one feature.
  - **OCP:** New features added without modifying existing classes.



6. State Pattern

- ★ **Role:** `OrderState` controls order lifecycle behavior.
- ★ **Problem Solved:** Replaces complex conditional logic with state-specific behavior.
- ★ **SOLID Compliance:**
  - **SRP:** Each state handles its own logic.
  - **OCP:** New states added without altering existing ones.



SOLID Principle Adherence and Quality Analysis

Using the six design patterns along with SOLID principles makes the Restaurant Management & Ordering System (RMOS) better and more reliable in the following ways:

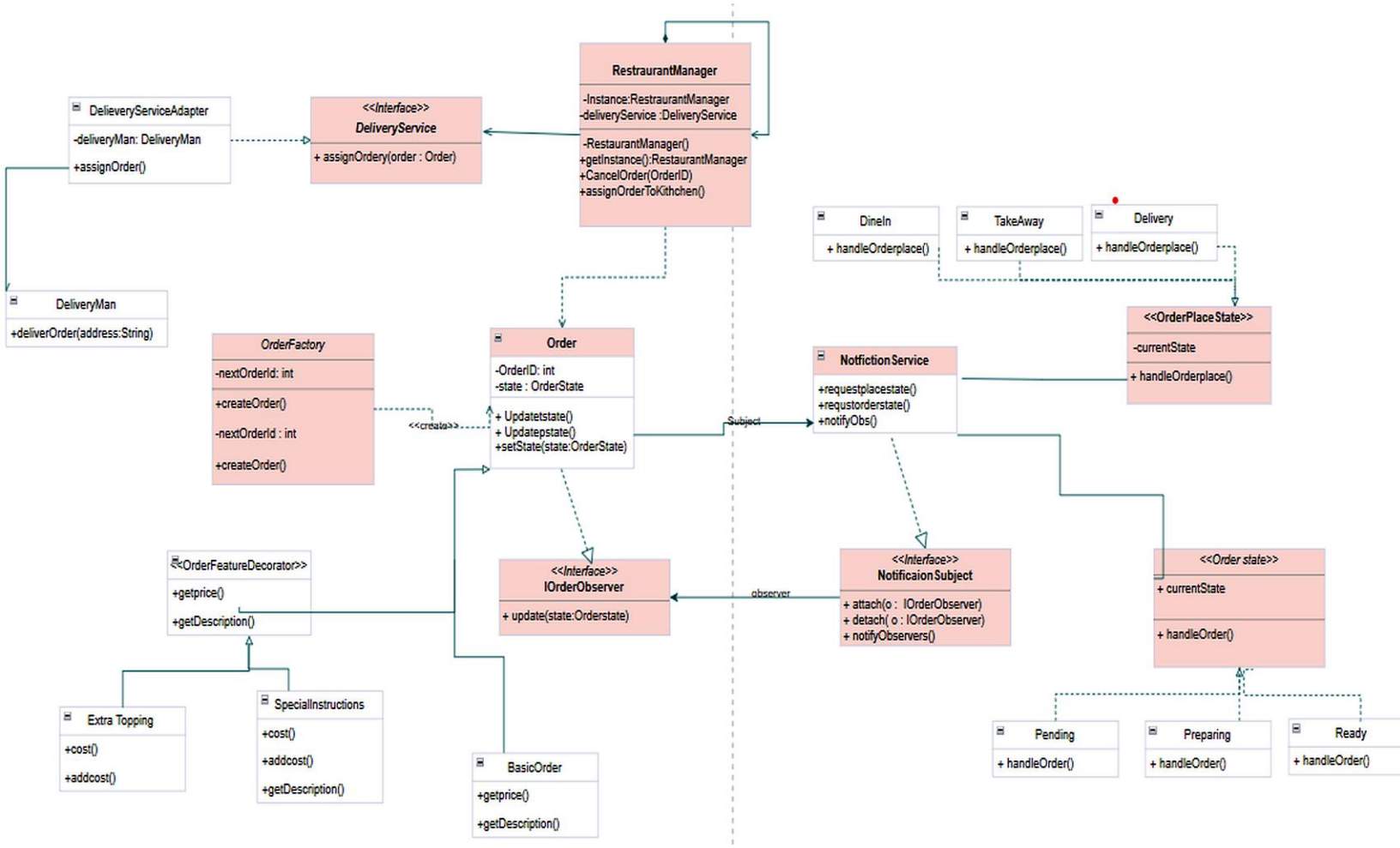
1. Code Reusability

- ★ The **Factory Pattern** lets the system create different types of orders without rewriting code.
- ★ The **Decorator Pattern** allows adding extra features (like toppings or combos) to orders easily.
- ★ Using interfaces and abstractions makes code reusable across different parts of the system.

2. Maintainability

- ★ Each class does only one job (**Single Responsibility Principle**), making the system easier to understand and change.
- ★ The **Observer Pattern** separates notifications from order and table management, so updates don't break other parts.
- ★ The **State Pattern** keeps order behavior organized, avoiding messy conditional statements.

3. Scalability

- ★ The system can grow easily because new order types, states, or delivery services can be added without changing existing code (**Open/Closed Principle**).
- ★ The **Adapter Pattern** allows connecting new delivery platforms smoothly.

★ The **Singleton Pattern** keeps system control consistent, even as more operations are added.